PRAKTIKUM BASIS DATA

MODUL 2

DDL DML

Oleh:

Geusan Edurais Aria Daffa 103122400026

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. **Buatlah setiap tabel diatas, tuliskan script SQL nya dan sertakan tangkapan layarnya**
   Jawaban:

```sql
CREATE TABLE FILM (
    id_film NUMBER PRIMARY KEY,
    judul VARCHAR2(100),
    sinopsis VARCHAR2(250),
    tahun NUMBER(4),
    durasi NUMBER
);

CREATE TABLE Theater (
    id_theater NUMBER PRIMARY KEY,
    harga NUMBER(10),
    kapasitas NUMBER(3),
    kelas VARCHAR2(20)
);

CREATE TABLE Member (
    id_member NUMBER PRIMARY KEY,
    nama VARCHAR2(100),
    no_hp VARCHAR2(15),
    tgl_lahir DATE,
    email VARCHAR2(50)
);

CREATE TABLE Jadwal (
    id_jadwal NUMBER PRIMARY KEY,
    id_film NUMBER,
    id_theater NUMBER,
    periode_start TIMESTAMP,
    periode_end TIMESTAMP,
    CONSTRAINT fk_jadwal_film FOREIGN KEY (id_film) REFERENCES Film(id_film),
    CONSTRAINT fk_jadwal_theater FOREIGN KEY (id_theater) REFERENCES Theater(id_theater)
);

CREATE TABLE Inventaris (
    id_inventaris NUMBER PRIMARY KEY,
    id_theater NUMBER,
    nomor_kursi VARCHAR2(5),
    CONSTRAINT fk_inv_theater FOREIGN KEY (id_theater) REFERENCES Theater(id_theater)
);

CREATE TABLE Transaksi (
    kode_pemesanan VARCHAR2(10) PRIMARY KEY,
    id_jadwal NUMBER,
    id_member NUMBER,
    status VARCHAR2(20),
    tanggal DATE,
    total_harga NUMBER(10),
    CONSTRAINT fk_trans_jadwal FOREIGN KEY (id_jadwal) REFERENCES Jadwal(id_jadwal),
    CONSTRAINT fk_trans_member FOREIGN KEY (id_member) REFERENCES Member(id_member)
);
```

Kode SQL tersebut membangun system database yang terdiri dari enam table saling terhubung melalui Primary Key dan Foreign Key. Tabel Film, Theater, dan Member berfungsi sebagai entitas utama yang menyimpan data. Data tersebut kemudian

dihubungkan dalam tabel Jadwal (menentukan waktu tayang film di studio tertentu) dan Inventaris (mengelola kursi studio).

2. **Lakukan insert data pada ke-enam tabel, sertakan tangkapan layar dan keterangannya**
   Jawaban:

   1. **Insert Table Film**

   Script:

```
INSERT INTO Film VALUES (1, 'Avengers', 'Superhero fight', 2019, 180);
INSERT INTO Film VALUES (2, 'Interstellar', 'Luar angkasa', 2014, 169);
INSERT INTO Film VALUES (3, 'John Wick', 'Action killer', 2014, 101);
```

   Output:

	ID_FILM	JUDUL	SINOPSIS	TAHUN	DURASI
1	1	Avengers	Superhero fight	2019	180
2	2	Interstellar	Luar angkasa	2014	169
3	3	John Wick	Action killer	2014	101

   Kode SQL tersebut merupakan perintah dari DML yang berfungsi untuk mengisi/menambahkan data awal ke dalam tabel film. tabel Film yang tadinya kosong sekarang memiliki data (seperti Avengers, Interstellar, dan John Wick) yang siap digunakan oleh tabel lain, seperti tabel Jadwal, untuk diatur waktu penayangannya di bioskop.

   2. **Insert Table Theater**

   Script:

```
INSERT INTO Theater VALUES (1, 50000, 50, 'Reguler');
INSERT INTO Theater VALUES (2, 75000, 30, 'VIP');
INSERT INTO Theater VALUES (3, 100000, 20, 'VVIP');
```

   Output:

	ID_THEATER	HARGA	KAPASITAS	KELAS
1	1	50000	50	Reguler
2	2	75000	30	VIP
3	3	100000	20	VVIP

Pada Tabel Theater diisi dengan 3 jenis kelas studio. Baris-baris ini menentukan kategori studio yang tersedia di bioskop dengan memasukkan data berupa nomor ID studio, harga tiket per kursi, total kapasitas penonton, hingga kelas studio. Dengan data ini, sistem memiliki acuan harga dan kapasitas yang nantinya akan digunakan oleh tabel Jadwal untuk menentukan lokasi tayang film dan oleh tabel Transaksi untuk menghitung total biaya yang harus dibayar oleh pelanggan berdasarkan kelas studio yang dipilih. Data berhasil masuk semua, terbukti dari outputnya yang nampillin 3 baris data.

3. Insert Table Member

Script:

```
INSERT INTO Member VALUES (1, 'Geusan', '08123', TO_DATE('2005-06-04','YYYY-MM-DD'), 'geusan@mail.com');
INSERT INTO Member VALUES (2, 'Rasyid', '08567', TO_DATE('2002-04-14','YYYY-MM-DD'), 'rasyid@mail.com');
INSERT INTO Member VALUES (3, 'Humaid', '08999', TO_DATE('2000-12-12','YYYY-MM-DD'), 'humaid@mail.com');
```

Output:

	ID_MEMBER	NAMA	NO_HP	TGL_LAHIR	EMAIL
1	1	Geusan	08123	04-06-2005	geusan@mail.com
2	2	Rasyid	08567	14-04-2002	rasyid@mail.com
3	3	Humaid	08999	12-12-2000	humaid@mail.com

Di tabel Member ini saya mendaftarkan tiga profil orang yang berbeda dengan mencatat ID unik, nama, nomor telepon, tanggal lahir, dan alamat email mereka. Fungsi TO_DATE digunakan agar format tanggal yang kamu tulis bisa dibaca dengan benar oleh sistem.

.

4. Insert Table Jadwal

Script:

```
INSERT INTO jadwal VALUES (
    1,
    1,
    1,
    TO_TIMESTAMP('2024-05-20 13:00:00', 'YYYY-MM-DD HH24:MI:SS'),
    TO_TIMESTAMP('2024-05-20 16:00:00', 'YYYY-MM-DD HH24:MI:SS')
);

INSERT INTO jadwal VALUES ( 2,
    2,
    2,
    TO_TIMESTAMP('2024-05-21 19:00:00', 'YYYY-MM-DD HH24:MI:SS'),
    TO_TIMESTAMP('2024-05-21 21:50:00', 'YYYY-MM-DD HH24:MI:SS')
);

INSERT INTO jadwal VALUES ( 3,
    3,
    3,
    TO_TIMESTAMP('2024-05-22 21:00:00', 'YYYY-MM-DD HH24:MI:SS'),
    TO_TIMESTAMP('2024-05-22 22:45:00', 'YYYY-MM-DD HH24:MI:SS')
);
```

Output:

	ID_JADWAL	ID_FILM	ID_THEATER	PERIODE_START	PERIODE_END
1	1	1	1	20-05-2024 13.00.00,000000000	20-05-2024 16.00.00,000000000
2	2	2	2	21-05-2024 19.00.00,000000000	21-05-2024 21.50.00,000000000
3	3	3	3	22-05-2024 21.00.00,000000000	22-05-2024 22.45.00,000000000

Tabel JADWAL berfungsi menghubungkan data film dan studio berdasarkan waktu tayang yang spesifik. Jadwal 1 mengatur Film ID 1 di Theater ID 1 pada 20 Mei 2024, Jadwal 2 untuk Film ID 2 di Theater ID 2 pada 21 Mei 2024, dan Jadwal 3 untuk Film ID 3 di Theater ID 3 pada 22 Mei 2024. Karena adanya aturan Foreign Key, jadi tidak bisa memasukkan ID sembarangan; data Film dan Theater harus sudah terdaftar lebih dulu agar sistem tidak error

5. **Insert Table Transaksi**

Script:

```
INSERT INTO Transaksi VALUES (1, 1, 1, 'Lunas', SYSDATE, 50000);
INSERT INTO Transaksi VALUES (2, 2, 2, 'Lunas', SYSDATE, 75000);
INSERT INTO Transaksi VALUES (3, 3, 3, 'Belum Bayar', SYSDATE, 100000);
```

Output:

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	1	1	1	Lunas	08-03-2026	50000
2	2	2	2	Lunas	08-03-2026	75000
3	3	3	3	Belum Bayar	08-03-2026	100000

Tabel TRANSAKSI berfungsi mencatat setiap pembelian tiket yang dilakukan oleh member. Transaksi 1 dan 2 berstatus 'Lunas' (Rp50.000 & Rp75.000), sementara Transaksi 3 berstatus 'Belum Bayar' (Rp1000.000). Karena adanya aturan Foreign Key, data ini hanya bisa diinput jika ID Jadwal dan ID Member sudah terdaftar sebelumnya di tabel masing-masing agar data tetap akurat.

6. **Insert Table Inventaris**

Script:

```
INSERT INTO Inventaris VALUES (1, 1, 'A1');
INSERT INTO Inventaris VALUES (2, 2, 'B1');
INSERT INTO Inventaris VALUES (3, 3, 'C1');
```

Output:

	ID_INVENTARIS	ID_THEATER	NOMOR_KURSI
1	1	1	A1
2	2	2	B1
3	3	3	C1

Tabel Inventaris berfungsi untuk mencatat detail fisik aset di setiap studio, khususnya penomoran kursi. Di sini, kursi 'A1' didaftarkan untuk Theater 1, 'B1' untuk Theater 2, dan 'C1' untuk Theater 3. Karena kolom id_theater merupakan Foreign Key, data ini hanya bisa diinput jika ID studio tersebut sudah terdaftar di tabel Theater agar posisi kursi tidak nyasar ke studio yang salah.

3. **Lakukan update data (minimal 5 data), tuliskan script SQL nya dan sertakan tangkapan layarnya**

1. **Update pada table film**

Script:

```
UPDATE Film SET durasi = 185 WHERE id_film = 1;
```

Output:

	ID_FILM	JUDUL	SINOPSIS	TAHUN	DURASI
1	1	Avengers	Superhero fight	2019	185
2	2	Interstellar	Luar angkasa	2014	169
3	3	John Wick	Action killer	2014	101

Perintah UPDATE ini digunakan untuk mengubah data spesifik pada tabel Film. Pada contoh tersebut, durasi film dengan id_film = 1 diperbarui menjadi 185 menit. Penggunaan WHERE sangat penting sebagai filter agar perubahan hanya terjadi pada satu baris yang dituju. Tanpa WHERE, semua durasi film di dalam tabel akan berubah secara massal, yang tentu saja akan merusak keakuratan data lainnya.

2. **Update pada table Theater**

Script:

```
UPDATE Theater SET harga = 60000 WHERE id_theater = 1;
UPDATE Theater SET kapasitas = 45 WHERE id_theater = 1;
```

Output:

	ID_THEATER	HARGA	KAPASITAS	KELAS
1	1	60000	45	Reguler
2	2	75000	30	VIP
3	3	100000	20	VVIP

Perintah UPDATE ini digunakan untuk memperbarui dua informasi sekaligus pada tabel Theater. Pada bagian ini, data harga tiket untuk studio nomor 1 dinaikkan menjadi 60000, dan kapasitas kursinya juga diperbarui menjadi 45. Hasil dari eksekusi ini adalah kolom harga dan kapasitas pada baris studio tersebut akan langsung tersinkronisasi dengan nilai yang baru.

3. **Update pada table Member**

Script:

```
UPDATE Member SET no_hp = '08238247234' WHERE id_member = 1;
```

Output:

	ID_MEMBER	NAMA	NO_HP	TGL_LAHIR	EMAIL
1	1	Geusan	08238247234	04-06-2005	geusan@mail.com
2	2	Rasyid	08567	14-04-2002	rasyid@mail.com
3	3	Humaid	08999	12-12-2000	humaid@mail.com

Perintah UPDATE ini digunakan untuk memperbarui informasi kontak pelanggan yang tersimpan di dalam tabel Member. Pada bagian ini, data nomor HP milik member dengan ID nomor 1 diperbarui menjadi '08238247234'. Hasil dari eksekusi ini adalah kolom no_hp pada baris member tersebut akan langsung tersinkronisasi dengan nomor telepon yang baru.

4. **Update pada table Transaksi**

Script:

```
UPDATE Transaksi SET status = 'Selesai' WHERE kode_pemesanan = 1;
```

Output:

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	1	1	1	Selesai	08-03-2026	50000
2	2	2	2	Lunas	08-03-2026	75000
3	3	3	3	Belum Bayar	08-03-2026	100000

Perintah UPDATE terakhir ini digunakan untuk memperbarui data operasional pada tabel Transaksi. Dalam skenario ini, kolom status untuk pesanan dengan kode pemesanan nomor 1 diubah nilainya menjadi 'Selesai'. Hasil dari eksekusi ini adalah status transaksi tersebut kini telah diperbarui di dalam database, yang menandakan bahwa proses pemesanan atau pembayaran untuk tiket tersebut sudah tuntas..

4. **Lakukan hapus data (minimal 2 data), tuliskan script SQL nya dan sertakan tangkapan layarnya**

1. **Delete inventaris dengan kode inventaris 1**

Script:

```
DELETE FROM Inventaris WHERE id_inventaris = 1;
```

Output:

	ID_INVENTARIS	ID_THEATER	NOMOR_KURSI
1	2	2	B1
2	3	3	C1

Perintah DELETE ini digunakan untuk menghapus data secara permanen dari tabel Inventaris. Pada perintah SQL tersebut, data kursi dengan id_inventaris = 1 dihapus dari sistem.

2. **Delete transaksi dengan kode pemesanan 1**

Script:

```
DELETE FROM Transaksi WHERE kode_pemesanan = 1;
```

Output:

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	2	2	2	Lunas	08-03-2026	75000
2	3	3	3	Belum Bayar	08-03-2026	100000

Perintah DELETE ini digunakan untuk menghapus data secara permanen dari tabel Transaksi. Pada perintah SQL tersebut, data kursi dengan kode_pemesanan= 1 dihapus dari sistem.